



Waipapa
Taumata Rau
**University
of Auckland**

DL Frameworks: Keras, TensorFlow, PyTorch, and Jax

SOFTENG 753: Machine Learning Techniques
and Applications

>>> Semester 2, 2026

>>> Maryam Hemmati
m.hemmati@auckland.ac.nz

Content



- **Major Deep Learning Frameworks & Their Relationship**
- **TensorFlow**
- **PyTorch**
- **Jax**
- **Keras**

Deep Learning Frameworks

- In the real world, we do not write low-level code from scratch.
 - Instead, we use a [Deep Learning Framework](#).
- The main deep learning frameworks today are
 - **Keras** (François Chollet, [2015](#))
 - **TensorFlow** (Google Brain, [2015](#))
 - **PyTorch** (Meta AI, [2016](#))
 - **JAX** (Google, [2018](#))
- All these frameworks are quite recent
 - [Keras](#) is the oldest among the four. It was launched in [March 2015](#).
- The ideas behind these frameworks have a long history
 - The [first paper](#) about [automatic differentiation](#) was published in [1964](#).

Deep Learning Frameworks

- All Deep Learning frameworks combine **three key features**
 - A way to **compute gradients** for arbitrary differentiable functions ([automatic differentiation](#))
 - A way to **run tensor computations** on [CPUs](#) and [GPUs](#) (and possibly on other specialized deep learning hardware)
 - A way to **distribute computation** across multiple devices or multiple computers
 - Multiple GPUs on one computer
 - Multiple GPUs across multiple separate computers

Deep Learning Frameworks

- **Keras** focuses on **high-level** deep learning concepts
 - Layers
 - Models
 - Optimizers
 - Losses
 - Metrics
 - Training Loops
- **TensorFlow, PyTorch, and JAX** focus on **tensor manipulation infrastructure**
 - Variables
 - Gradients
 - Automatic differentiation
 - Distribution

TensorFlow

- There are a few different ways we can create tensors in TensorFlow.
 - **Constant tensors**
 - `tf.ones()`
 - `tf.zeros()`
 - `tf.constant()`
 - **Random tensors**
 - `tf.random()`
- A **significant difference** between NumPy arrays and TensorFlow tensors is that TensorFlow tensors are **not assignable**
 - **TensorFlow tensors are constant.**
- To train a model, we need to **update** its **state**, which is a set of tensors.
 - Tensors are not assignable.
 - We will use **variables**.
 - **`tf.Variable`** is the class meant to manage **modifiable state** in TensorFlow.

TensorFlow

- To **create** a variable, we need to provide some **initial value**, such as a random tensor.

```
>>> v = tf.Variable(initial_value=tf.random.normal(shape=(3, 1)))
>>> print(v)
array([[ -0.75133973],
       [-0.4872893 ],
       [ 1.6626885 ]], dtype=float32)>
```

- The state of a variable can be **modified** via its **assign()** method.

```
>>> v.assign(tf.ones((3, 1)))
array([[1.],
       [1.],
       [1.]], dtype=float32)>
```

- Assignment also works for a **subset** of the coefficients.

```
>>> v[0, 0].assign(3.)
array([[3.],
       [1.],
       [1.]], dtype=float32)>
```

TensorFlow

- NumPy **cannot** retrieve the **gradient of any differentiable expression** with respect to any of its inputs.
 - **TensorFlow can!**
- We can open a **GradientTape scope**
 - Apply some computation to one or several input tensors.
 - Retrieve the gradient of the result with respect to the inputs.
- Most commonly used to retrieve the **gradients of the loss** of a model with respect to its weights:
gradients = tape.gradient(loss, weights)
- The **gradient tape** is a powerful utility and is capable of **computing second-order gradients**.

TensorFlow

➤ Eager execution

- Operations are executed **one after the other** in the Python runtime.
- Like any Python code or NumPy code.

➤ Eager execution is **great for debugging**, but it is typically quite **slow**.

➤ It will be beneficial to

- **Parallelize** some computation
- **Fuse** operations
 - Replacing two consecutive operations with a single, more efficient operation that does the same thing without materializing the intermediate output.

TensorFlow

➤ Compilation

- The main benefit of compilation is **improved performance**.
- Its drawback is more painful debugging experience.
- Only turn on compilation after code is already debugged in the Python runtime.

➤ We can apply **compilation** to any TensorFlow function by wrapping it in a **tf.function** decorator.

➤ TensorFlow has **two compilation modes**:

- **First**, the default one, which we refer to as **"graph mode"**.
 - Any function decorated with **@tf.function** runs in graph mode.
- **Second**, compilation with XLA (Accelerated Linear Algebra)
 - XLA is a high-performance compiler for ML
 - We can turn it on by specifying **jit_compile=True**, like this:

TensorFlow

TensorFlow ecosystem for production deployment is the most mature among all frameworks.

Let's look at a short clip about TensorFlow

[TensorFlow in 100 Seconds](#)



TensorFlow

PyTorch

- The **PyTorch** API is **higher level** than that of TensorFlow and JAX
 - it includes **layers** and **optimizers**, like Keras.
 - These layers and optimizers are **compatible** with **Keras** workflows when you use Keras with the PyTorch backend.
- Random tensor creation is similar to NumPy and TensorFlow, but with **divergent syntax**.
 - The function **normal** does not take a shape argument.
 - Instead, the **mean** and **standard deviation** should be provided as PyTorch tensors with the **expected output shape**.
- Like NumPy arrays, PyTorch tensors are **assignable**.
- Math in PyTorch works just the **same** as math in NumPy or TensorFlow.

PyTorch

- When we run any computation in PyTorch, the framework creates a **one-time computation graph** that records what just happened.
 - There's no **explicit** "gradient tape".
 - The tape is hidden from the user.
- The public API for using it is at the level of tensors themselves.
 - We can call **tensor.backward()** to run backpropagation through all operations previously executed that led to that tensor.
 - Doing this will populate the **.grad** attribute of all tensors that are tracking gradients.
- If we call **backward()** multiple times in a row, the **.grad** attribute will **accumulate** gradients
 - Each new call will sum the new gradient with the preexisting one.

PyTorch

- PyTorch lets us compile **functions** or even **Module** instances via the **torch.compile()** utility.
 - This API uses PyTorch's **own compiler**, named **Dynamo**.
- PyTorch code **executes eagerly by default**, making it **easy to debug**.
 - Due to its focus on eager execution, PyTorch is **quite slow**.
 - It is the **slowest of all** the major frameworks by a large margin.

PyTorch

The popular pretrained model-sharing platform Hugging Face has first-class support for PyTorch.

Let's look at a short clip about PyTorch
[PyTorch in 100 Seconds](#)



JAX

- JAX quickly gained traction in the research community, particularly for its ability to use [Google's TPUs at scale](#).
- JAX is in use by most of the [top players](#) in the [generative AI](#) space
 - Companies like DeepMind, Apple, Midjourney, Anthropic, Cohere, and so on.
- JAX embraces a [stateless](#) approach to computation
 - Functions in JAX do not maintain any persistent state.
 - This contrasts with [traditional imperative](#) programming, where variables can hold values between function calls.
- The [stateless](#) nature of JAX functions has several advantages.
 - It enables effective [automatic parallelization](#) and [distributed computation](#).
 - Functions can be executed [independently](#) without the need for [synchronization](#).

JAX

- One of the **best features** of JAX is that it doesn't try to implement its own independent
 - Instead, it just implements the **NumPy API**, as is.
- Doing **math** in JAX looks **exactly the same** as it does in NumPy.
- JAX takes a **metaprogramming** approach to gradient computation.
 - Metaprogramming refers to the idea of **having functions that return functions**.
 - JAX lets us turn a loss-computation function into a gradient-computation function.
- We can use the **@jax.jit** decorator, which behaves identically to the **@tf.function(jit_compile=True)** decorator.
 - It turns any stateless JAX function into an **XLA-compiled** piece of code.
 - Delivering a considerable **execution speedup**.

JAX

JAX is the best fit for training models on TPUs, as it was developed from the ground up for XLA and TPUs.

Let's look at a short clip about JAX

JAX in 100 Seconds

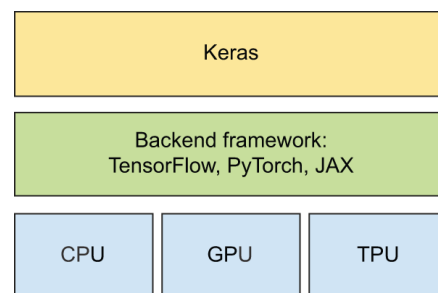


Deep Learning Frameworks

Let's have a look and try different examples in [Jupyter notebook](#)

Keras

- Backend frameworks are **pluggable**
 - We can switch to a different backend framework after we have written some **Keras** code.
 - We are not locked into a single framework and a single ecosystem
 - We can move our models from **JAX** to **TensorFlow** to **PyTorch** depending on our current needs.
- We can **develop** a **Keras model**, and
 - **Debug** it with **PyTorch**
 - **Train** it on TPU with **JAX** for maximum efficiency
 - **Run inference** with the excellent tooling from the **TensorFlow** ecosystem.



Deep learning development: layers, models, optimizers, losses, metrics, training loops ...

Tensor manipulation infrastructure: tensors, variables, automatic differentiation, distribution ...

Hardware: execution

Keras and its backends. A backend is a low-level tensor-computing platform; Keras is a **high-level** deep learning API.

Keras – Configuring Backend

- Keras is not a fully standalone framework. It needs a **backend engine** to run.
- The **default backend** for Keras right now is **TensorFlow**
 - Running `import keras` in a fresh environment, without having configured anything, results in running on top of **TensorFlow**.
- There are two ways to pick a different backend
 - Set the environment variable **KERAS_BACKEND**.
 - Edit the local Keras **configuration file**.

Keras – Configuring Backend

Set the environment variable **KERAS_BACKEND**

- Run the following shell command before starting the python repl
 - `export KERAS_BACKEND=jax`
 - `export KERAS_BACKEND=torch`
 - `export KERAS_BACKEND=tensorflow`
- Add the following code snippet at the top of the Python file or notebook

```
import os

# Sets the environment variable from within the Python runtime
os.environ["KERAS_BACKEND"] = "jax"

# Only then should you import Keras.
import keras
```

Use the string **"torch"** to refer to the PyTorch backend, rather than the string **"pytorch"**, which would be **invalid**.

- It must imperatively go **before the first `import keras`**

Keras – Configuring Backend

Edit the local Keras configuration file

- If Keras has been imported once, the file is already created with default settings.
- It is a human-readable JSON file.

```
{
  # Default floating-point precision. It should typically not be
  # changed.
  "floatx": "float32",
  # Default numerical fuzzing factor. It should typically not be
  # changed.
  "epsilon": 1e-07,
  # Change "tensorflow" to "jax" or "torch."
  "backend": "tensorflow",
  # This is the default image layout. We'll talk about this in
  # chapter 8.
  "image_data_format": "channels_last",
}
```

Use the string `"torch"` because PyTorch package name is `torch`.

Keras Layers

- The fundamental data structure in neural networks is the **layer**.
- A **layer** is a **data processing module** that takes one or more tensors as **input** and **outputs** one or more tensors.
- Layers usually have a **state** which is the layer's **weights**.
 - One or several tensors **learned** with stochastic gradient descent
 - Together they contain the **network's knowledge**.
- Different types of layers are appropriate for different **tensor formats** and different **types of data** processing.
 - **Simple vector data**, stored in 2D tensors of shape (**samples, features**), is often processed by densely connected layers, also called **fully connected** or **dense layers** (the **Dense** class in Keras).
 - **Sequence data**, stored in 3D tensors of shape (**samples, timesteps, features**), is typically processed by recurrent layers, such as an **LSTM layer**, or **1D convolution layers** (**Conv1D**).
 - **Image data**, stored in rank-4 tensors, is usually processed by **2D convolution layers** (**Conv2D**).

Keras Layers

- A metaphor made explicit by Keras:

Think of *layers* as the LEGO bricks of deep learning.

- Building deep learning models in Keras is done by *clipping* together *compatible* layers to form useful *data transformation* pipelines.

Keras Layers

- The *Layer class* is a single abstraction around which everything is centered.
 - Everything in Keras is either a *Layer* or something that closely interacts with a *Layer*.
- A Layer is an object that encapsulates
 - Some **state** (*weights*)
 - Some **computation** (*a forward pass*)
- The weights are typically defined in a *build()* method.
 - Could also be created in the constructor *__init__()*
- The computation is defined in the *call()* method.

Keras Layers - Compatibility

- Just like with LEGO bricks, we can only *clip* together layers that are compatible.
- Every layer
 - will **only accept** input tensors of a **certain shape**.
 - will **return** output tensors of a **certain shape**.
- Using Keras, we don't have to worry about size compatibility most of the time
 - The layers we add to our models are dynamically built to match the shape of the incoming inputs.

```
from keras import layers

# A dense layer with 32 output units
layer = layers.Dense(32, activation="relu")
```

This layer can **only** be connected to a downstream layer that expects **32-dimensional** vectors as its **input**.

```
from keras import models
from keras import layers

model = models.Sequential(
    [
        layers.Dense(32, activation="relu"),
        layers.Dense(32),
    ]
)
```

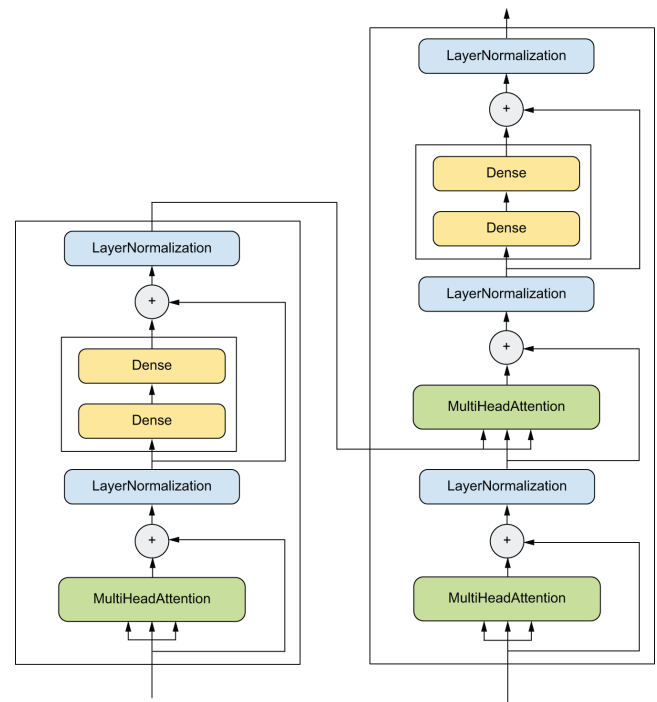
Layers **automatically infer** their **input shape** as the shape of the first inputs they see.

From Layers To Models

- A deep learning **model** is a **graph of layers**.
 - The **Model class** in Keras
- **Sequential models** (a **subclass** of Model), are simple stacks of layers
 - Mapping a **single input** to a **single output**.
- A much **broader variety** of network topologies exist
 - **Two-branch** networks
 - **Multihead** networks
 - **Residual** connections

Keras Models

- The **topology** of a model defines a **hypothesis space**.
- By choosing a network topology, we **constrain** our space of possibilities (hypothesis space) to
 - A **specific series of tensor operations**, mapping input data to output data.
- The structure of our hypothesis space (the **architecture of our model**) is **extremely important**.
 - It encodes the **assumptions** we make about our problem



The **Transformer** architecture.

Keras Models - The "Compile" Step

- Once the model architecture is defined, we have to choose **three more things**:
 - **Loss function** (objective function)
 - The **quantity** that will be **minimized** during training.
 - It represents a measure of success for the task at hand.
 - **Optimizer**
 - Determines how the network will be **updated** based on the **loss function**.
 - It implements a specific variant of stochastic gradient descent (SGD).
 - **Metrics**
 - The measures of **success** to monitor training and validation, such as classification accuracy.
 - Training will not optimize directly for these metrics.
 - Metrics **do not need** to be differentiable.

Keras Models - The "Compile" Step

- After picking our loss, optimizer, and metrics, we use the built-in `compile()` and `fit()` methods to start training your model.
- The `compile()` method configures the training process.
- We can pass the optimizer, loss, and metrics as `strings` (such as `"rmsprop"`).
 - These strings are shortcuts that get converted to `Python objects`.

```
# Defines a linear classifier
model = keras.Sequential([keras.layers.Dense(1)])
model.compile(
    # Specifies the optimizer by name: RMSprop (it's case-insensitive)
    optimizer="rmsprop",
    # Specifies the loss by name: mean squared error
    loss="mean_squared_error",
    # Specifies a list of metrics: in this case, only accuracy
    metrics=["accuracy"],
)
```

Keras Models - The "Compile" Step

- It is possible to specify these arguments as `object instances`.

```
model.compile(
    optimizer=keras.optimizers.RMSprop(),
    loss=keras.losses.MeanSquaredError(),
    metrics=[keras.metrics.BinaryAccuracy()],
)
```

- This is useful if
 - We want to pass your `own custom` losses or metrics
 - We want to further `configure` the objects we are using. For instance, by passing a `learning_rate` argument to the optimizer

```
model.compile(
    optimizer=keras.optimizers.RMSprop(learning_rate=1e-4),
    loss=my_custom_loss,
    metrics=[my_custom_metric_1, my_custom_metric_2],
)
```


Keras Models - The Loss Function

- Choosing the **right loss function** for the **right problem** is extremely important
 - The network will take any shortcut it can to minimize the loss.
 - If the objective doesn't **fully correlate** with **success** for the task at hand, the network will end up doing things we may not have wanted.
- Imagine a stupid, omnipotent AI trained via SGD, with this **poorly chosen objective function**:
"Maximize the average well-being of all humans alive."
- To make its job easier, this AI might choose to **kill all humans except a few** and focus on the well-being of the remaining ones.
- **Average** well-being is **not affected** by how many humans are left!
That is not what we intended!

Keras Models - The "Fit" Step

- The **fit()** method implements the **training loop** itself.
- Its key arguments are
 - **The data** (inputs and targets) to train on.
 - It will typically be passed either in the form of **NumPy arrays** or a **TensorFlow Dataset** object.
 - **The number of epochs** to train for
 - How **many times** the training loop should **iterate** over the data passed.
 - **The batch size** to use within each epoch of mini-batch gradient descent
 - The number of **training examples** considered to compute the gradients for **one weight update step**.

Keras Models - The "Fit" Step

- The call to `fit` returns a `History` object.
- This object contains a `history` field, which is a `dict mapping key`, such as `"loss"` or specific metric names to the list of their per-epoch values

```
history = model.fit(  
    # The input examples, as a NumPy array  
    inputs,  
    # The corresponding training targets, as a NumPy array  
    targets,  
    # The training loop will iterate over the data 5 times.  
    epochs=5,  
    # The training loop will iterate over the data in batches of 128  
    # examples.  
    batch_size=128,  
)
```

```
>>> history.history  
{  
  "binary_accuracy": [0.855, 0.9565, 0.9555, 0.95, 0.951],  
  "loss": [0.6573270302042366,  
           0.07434618508815766,  
           0.07687718723714351,  
           0.07412414988875389,  
           0.07617757616937161]}
```

Keras Models – Validation

- The goal of machine learning is to obtain `models that perform well in general`, particularly on data points that the `model has never encountered` before.
- A model performs well on its training data **does not mean** it will perform well on data it has never seen!
 - It is possible that our model could end up merely `memorizing a mapping` between your training samples and their targets.
 - It will be `useless` for the task of `predicting targets` for data the model has never seen before.
- It is **standard practice** to `reserve a subset` of the training data as **"validation data"**
 - We won't be training the model on this data, but we will use it to `compute a loss value` and `metrics value`.
 - We do this by using the `validation_data` argument in `fit()`.
 - Like the training data, the validation data could be passed as `NumPy arrays` or as a `TensorFlow Dataset` object.

Keras Models – Validation

- The value of the loss on the validation data is called the *validation loss*, to distinguish it from the *training loss*.
- If we want to compute the validation loss and metrics *after training* is complete, we can call the *evaluate* method

```
loss_and_metrics = model.evaluate(val_inputs, val_targets, batch_size=128)
```

- The *evaluate()* method will iterate in batches (of size *batch_size*) over the data passed.
- It returns a list of scalars
 - The first entry is the *validation loss*.
 - The following entries are the *validation metrics*.

Keras Models – Inference

➤ Inference

- After training our model, we use it to make *predictions* on *new data*.

```
# Takes a NumPy array or a tensor for your current backend and returns  
# a tensor for your current backend  
predictions = model(new_inputs)
```

- A better way to do inference is to use the *predict()* method.
 - It will iterate over the data in *small batches* and return a NumPy array of predictions.

```
# Takes a NumPy array or a Dataset and returns a NumPy array  
predictions = model.predict(new_inputs, batch_size=128)
```



Summary

- We looked at different Deep Learning frameworks and discussed their features and relationship.
- We looked at Keras as a high-level Deep Learning framework.
- We looked at different methods used in the training loop and inference stages in Keras.



References

- The content and figures used in these slides are extracted from the textbook **Deep Learning with Python, Third Edition - François Chollet.**

Next Session: Classification

Any Questions?